

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on laboratory work no. 4

Operating Systems

Topic: The xv6 Shell from the Outside

Done by:
st. gr. FAF-242, **Vasile Brînză**

Checked by:
Reițman Patricia, *university assistant*
DISA, FCIM, UTM

Chișinău 2026

CONTENTS

PURPOSE OF THE WORK **3**
1 FIX SLEEP FOR THE CURRENT XV6 **4**
1.1 Which name does the tree use? 4
1.2 The updated sleep.c 4
1.3 Build and test 5
2 BASH VERSUS THE XV6 SHELL **6**
2.1 Running the same commands in both shells 6
2.2 Comparison table 7
2.3 ls after cd d 8
2.4 Answers to the Observe questions 9
3 WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV **11**
3.1 args.c for xv6 and args_linux.c for Linux 11
3.2 Running args in both shells 11
3.3 Answers to the Observe questions 13
4 READ THE SHELL **15**
4.1 The kinds of command 15
4.2 Where each kind is run 15
4.3 The code that handles cd 16
4.4 Answers to the Observe questions 16
CONCLUSIONS **18**
BIBLIOGRAPHY **19**

PURPOSE OF THE WORK

The purpose of this laboratory work is to understand what a command shell has to do, by using the xv6 shell (user/sh.c, about 500 lines) side by side with bash, following the laboratory task [1]. By the end of the work the goals are:

- to bring the sleep program from Lab 2 up to date with the current xv6, where the system call sleep() was renamed to pause();
- to run the same command lines in bash and in the xv6 shell and to explain every difference: redirection, pipes, grouping, sequences, background jobs, globbing, variables, quotes, PATH, history and completion;
- to write a small program that prints its argc and argv, for xv6 and for Linux, and to use it to find out who expands * and who removes the quotes;
- to read the source of the xv6 shell and find the five kinds of command it knows, where each of them is run, and how cd is handled.

The work was done on a Lenovo laptop running Linux Mint, in bash, with xv6-riscv [2] in ~/xv6-riscv, built with the RISC-V toolchain and run in QEMU (`make qemu`). The bash experiments were done in the test folder ~/shtest, which contained a file named README, as required. The files for this lab (sleep.c, args.c, args_linux.c and the Makefile) are in the lab4/ folder of the course repository os-2026-VasileBrinza [3].

1 FIX SLEEP FOR THE CURRENT XV6

1.1 Which name does the tree use?

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

In August 2025 the xv6 authors renamed the system call `sleep()` to `pause()`. The grep shows that user/user.h declares `int pause(int);` on line 25 – this is the current name of the system call – and an additional `int sleep(int);` declaration on line 28, kept in this tree from the earlier work (Figure 1.1). Because `pause` is declared, the program was updated to call `pause()`, as the task requires.

```
kynex@lenovo:~/xv6-riscv$ pwd
/home/kynex/xv6-riscv
kynex@lenovo:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
28:int sleep(int);
kynex@lenovo:~/xv6-riscv$
```

Figure 1.1 – The grep result in user.h

1.2 The updated sleep.c

The program checks that it received exactly one argument; otherwise it prints the usage message to standard error (file descriptor 2) and exits with code 1. Then it converts the argument from text to a number with `atoi()` and calls `pause()`, which suspends the process for that many clock ticks. `$U/_sleep\` is on line 153 of the UPROGS list in the Makefile, so the program is compiled and copied into the xv6 file system image.

```
kynex@lenovo:~/xv6-riscv$ nano user/sleep.c
kynex@lenovo:~/xv6-riscv$ nano user/sleep.c
kynex@lenovo:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }

    pause(atoi(argv[1]));
    // was sleep() before Aug 2025

    exit(0);
}
kynex@lenovo:~/xv6-riscv$
```

Figure 1.2 – The updated sleep.c

1.3 Build and test

```
make qemu
$ sleep 20
$ sleep
usage: sleep <ticks>
```

`make qemu` compiled user/sleep.c, linked it into user/_sleep, rebuilt fs.img with the new program and booted the kernel (Figure 1.3). `sleep 20` printed nothing and returned the prompt after about two seconds (one xv6 tick is about 0.1 s in QEMU); `sleep 10` and `sleep 1` returned faster. `sleep` without an argument printed **usage: sleep <ticks>**. The lines "exec failed" in the same figure are not caused by sleep: they appeared when the arrow keys were pressed at the xv6 prompt (see the history row of Table 2.1). QEMU was closed with Ctrl-A, X.

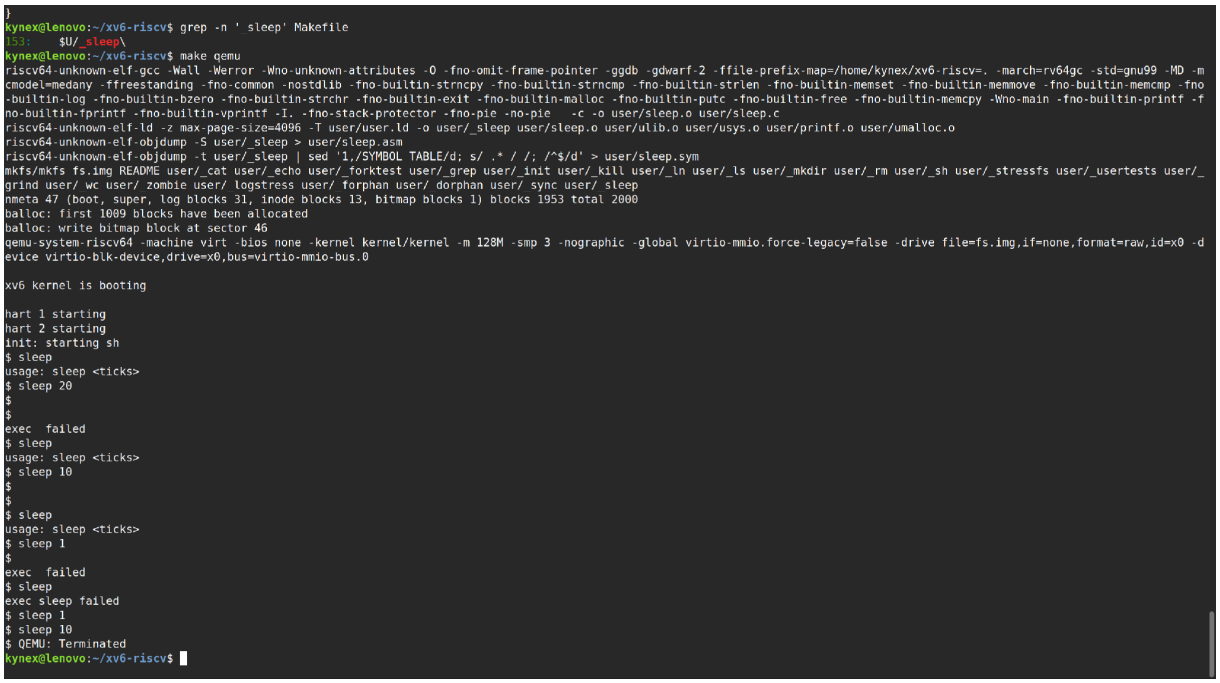


Figure 1.3 – sleep running in xv6, with and without an argument

2 BASH VERSUS THE XV6 SHELL

2.1 Running the same commands in both shells

```
mkdir ~/shtest && cd ~/shtest && touch README
```

The xv6 shell is described in chapter 1 of the xv6 book as an ordinary user program that reads commands and runs them with fork, exec and wait [4]. Every command of the task was run once in bash, in ~/shtest (Figure 2.1), and once at the xv6 $ prompt (Figure 2.2). In xv6 the line `mkdir d ; cd d ; ls .. ` was typed as three separate commands, and the quotes test was run as `echo "a" "b"` in both shells. Table 2.1 shows what each shell printed and why.

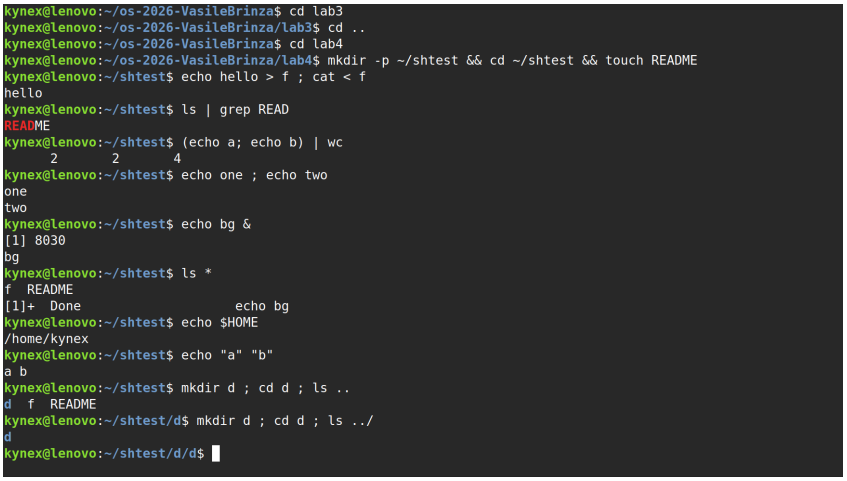


Figure 2.1 – The commands run in bash

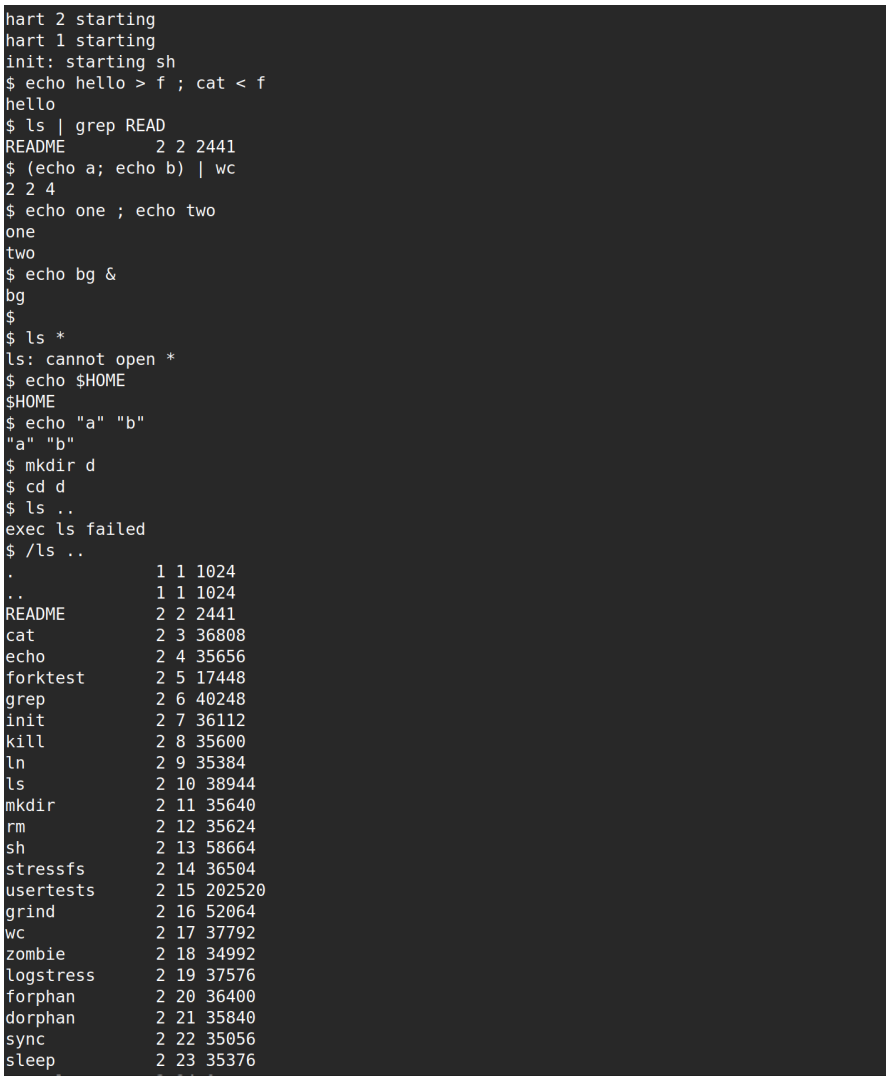


Figure 2.2 – The same commands run in xv6

2.2 Comparison table

Table 2.1 – bash versus the xv6 shell

Command	Tests	bash	xv6
`echo hello > f ; cat < f`	redirection	`hello` The shell opens f for writing for echo, then opens f as the standard input of cat.	`hello` Same result: sh.c supports <, > and >> (REDIR command).
`ls | grep READ`	pipe	`README` ls writes one name per line into the pipe, grep keeps the matching line.	`README 2 2 2441` The pipe works too; xv6 ls prints name, type, inode number and size on one line, so grep shows the whole line.
`(echo a; echo b) | wc`	grouping, list	`2 2 4` The subshell output (2 lines, 2 words, 4 bytes) goes into wc.	`2 2 4` Same: ( ) groups a list, and the group is the left side of the pipe.
`echo one ; echo two`	sequence	`one` / `two`	`one` / `two` Same: ; runs the commands one after another (LIST).

Command	Tests	bash	xv6
`echo bg &`	background	`[1] 8030`, `bg`, later `[1]+ Done echo bg` bash has job control: job number, PID and a Done message.	`bg`, then an extra `$` The command runs in the background (BACK), but there are no job numbers, no jobs/fg and no Done message; the prompt and the output can mix.
`ls *`	globbing	`f README` bash replaces * with the file names before ls starts.	`ls: cannot open *` No globbing: ls receives the literal argument "*".
`echo $HOME`	variables	`/home/kynex` bash substitutes the variable.	`$HOME` No variables (xv6 has no environment), the text is printed as typed.
`echo "a" "b"`	quotes	`a b` bash removes the quotes.	`"a" "b"` Quotes are ordinary characters for sh.c; they are not removed and do not group words.
`mkdir d ; cd d ; ls ..`	PATH	`d f README` ls is found through PATH (/usr/bin/ls) from any directory.	`exec ls failed` There is no PATH: exec looks for the file "ls" in the current directory d. `/ls ..` works.
arrow up, Tab, Ctrl+R	history, completion	Up recalls the previous command, Tab completes names, Ctrl+R searches the history (readline).	Nothing of this exists: the arrow key sends escape characters that become part of the line ("exec failed" in Figure 1.3); only Backspace and Ctrl-U editing is done by the console.

2.3 ls after cd d

In xv6, after `cd d` the command `ls ..` printed **exec ls failed** (Figure 2.3). Running it with an absolute path, `/ls ..`, worked and listed the root directory: the programs README, cat, echo, ..., ls, sh, sleep, the console device, the file f and the directory d. Another way is the relative path `../ls ..`.

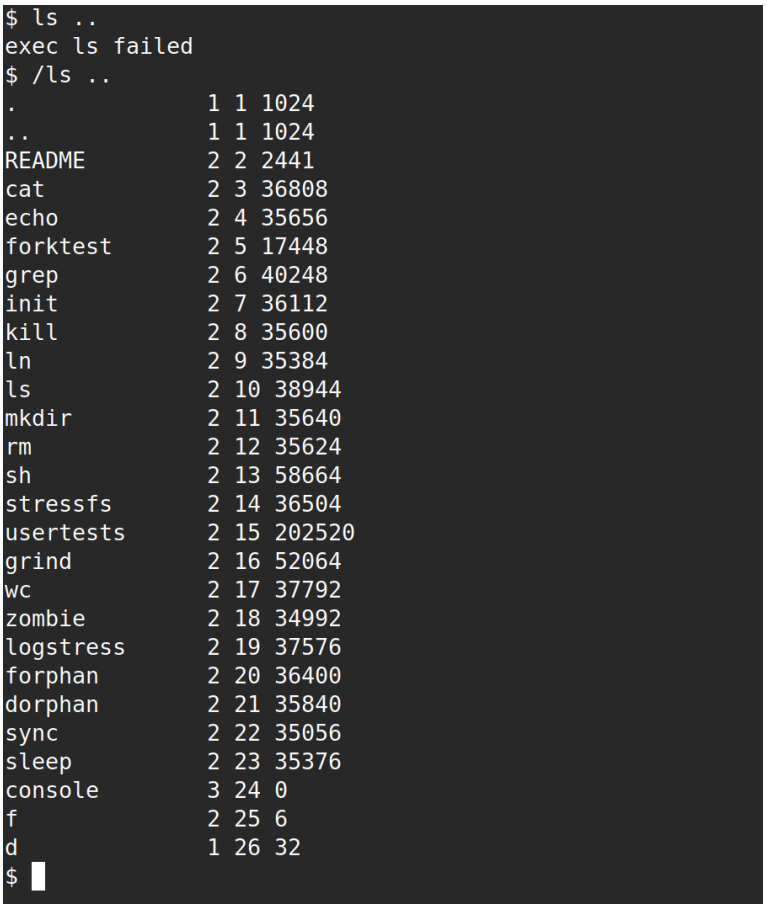


Figure 2.3 – ls failing and the working alternative

2.4 Answers to the Observe questions

1. List the features both shells have, and the ones only bash has.

Both shells: running a program with arguments, input and output redirection (<, >, >>), pipes (|), sequences (;), background commands (&), grouping with parentheses, and the built-in cd.

Only bash: filename globbing (*, ?, [ ]), variables and the environment ($HOME), quote removal (" " and ' '), the PATH search for programs, job control (job numbers, jobs, fg, Done messages), && and ||, command substitution, history, Tab completion and line editing (arrows, Ctrl+R), scripts with if/for/while, and many more builtins [5].

2. In xv6, ls stops working after cd d. Why? Find a way to run ls from inside d anyway.

All xv6 programs live in the root directory / of the file system (mkfs copies them there from UPROGS). The xv6 shell has no PATH: it calls `exec(argv[0], argv)` with the name exactly as typed, and the kernel resolves a name without "/" relative to the current directory. In / the name "ls" means /ls, so it works; inside /d it means /d/ls, which does not exist, so exec fails and the shell prints "exec ls failed". The fix is to give the path explicitly: `/ls ..` (absolute) or `../ls ..` (relative) – Figure 2.3.

3. Which missing feature would you miss most in daily work, and which would be easiest to add?

In daily work I would miss the command history with the up arrow and Tab completion the most, because they save the most typing, together with globbing. The easiest one to add is a PATH-like search: in runcmd, when `exec(argv[0], ...)` fails and the name does not contain "/", the shell can try again with "/" in front of the name. That is only a few lines of code. Simple $ variables or history would need much more work (an environment table, and console line editing).

3 WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV

3.1 args.c for xv6 and args_linux.c for Linux

user/args.c prints argc and then every element of argv between square brackets, so that spaces and quotes inside an argument are visible. `$U/_args\` was added at the end of UPROGS, after `$U/_sleep\` (Figure 3.1). args_linux.c is the same program with `#include <stdio.h>` and `return 0`; it was created with a here-document and compiled with `gcc -o args args_linux.c` in ~/xv6-riscv (Figure 3.3).

```
kynex@lenovo:~/xv6-riscv$ nano Makefile
kynex@lenovo:~/xv6-riscv$ cat user/args.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

```
UPROGS=\
	$U/_cat\
	$U/_echo\
	$U/_forktest\
	$U/_grep\
	$U/_init\
	$U/_kill\
	$U/_ln\
	$U/_ls\
	$U/_mkdir\
	$U/_rm\
	$U/_sh\
	$U/_stressfs\
	$U/_usertests\
	$U/_grind\
	$U/_wc\
	$U/_zombie\
	$U/_logstress\
	$U/_forphan\
	$U/_dorphan\
	$U/_sync\
	$U/_sleep\
	$U/_args\
```

Figure 3.1 – args.c and the UPROGS line

3.2 Running args in both shells

```
args one   two   three
args *.c "x y"
args $HOME
args 'single quoted'
```

In xv6 the program was run from the $ prompt (Figure 3.2) and in bash as ./args (Figure 3.3). The bash run was done in ~/xv6-riscv, where the only .c file in the folder itself is args_linux.c (the other sources are in kernel/ and user/). Table 3.1 summarizes the results.

Table 3.1 – Arguments received in xv6 and in bash

Command line	xv6 sh	bash
`args *.c "x y"`	argc = 4 `[args] [*.c] ["x] [y"]`	argc = 3 `[./args] [args_linux.c] [x y]`
`args $HOME`	argc = 2 `[args] [$HOME]`	argc = 2 `[./args] [/home/kynex]`
`args 'single quoted'`	argc = 3 `[args] ['single] [quoted']`	argc = 2 `[./args] [single quoted]`
`args one two three`	argc = 4 `[args] [one] [two] [three]`	argc = 2 for each word, run one at a time (Figure 3.3)

```
hart 2 starting
hart 1 starting
init: starting sh
$ args one
argc = 2
argv[0] = [args]
argv[1] = [one]
$ args two
argc = 2
argv[0] = [args]
argv[1] = [two]
$ args three
argc = 2
argv[0] = [args]
argv[1] = [three]
$ args one two three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
$
```

Figure 3.2 – args output in xv6

```
kynex@lenovo:~/xv6-riscv$ cat << 'EOF' > args_linux.c
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
EOF
kynex@lenovo:~/xv6-riscv$ gcc -o args args_linux.c
kynex@lenovo:~/xv6-riscv$ ./args one
argc = 2
argv[0] = [./args]
argv[1] = [one]
kynex@lenovo:~/xv6-riscv$ ./args two
argc = 2
argv[0] = [./args]
argv[1] = [two]
kynex@lenovo:~/xv6-riscv$ ./args three
argc = 2
argv[0] = [./args]
argv[1] = [three]
kynex@lenovo:~/xv6-riscv$ ./args *.c "x y"
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]
kynex@lenovo:~/xv6-riscv$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/kynex]
kynex@lenovo:~/xv6-riscv$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
kynex@lenovo:~/xv6-riscv$
```

Figure 3.3 – args_linux output in bash

3.3 Answers to the Observe questions

1. Paste both outputs for args *.c "x y". How many arguments did each program receive?

xv6: `argc = 4 — argv[0] = [args], argv[1] = [*.c], argv[2] = ["x], argv[3] = [y"]`, so the program received **3 arguments** besides its name: the pattern unchanged and the quoted string cut in two, with the quote characters still inside.

bash: `argc = 3 — argv[0] = [./args], argv[1] = [args_linux.c], argv[2] = [x y]`, so the program received **2 arguments**: the file name that matched *.c and the string x y as one argument, without quotes.

2. So who turns *.c into file names and removes the quotes: the program, the kernel, or the shell?

The shell. The program is identical on both systems and only prints what it receives, and exec in the kernel copies the argv array as it is given. The only thing that changed between the two runs is the shell: bash expands *.c, $HOME and removes the quotes before it calls exec, while the xv6 shell only splits the line at whitespace and passes the words unchanged. In POSIX terms these are the word expansions and quote removal that a shell performs before running a command [6].

3. What happened to the extra spaces in one two three?

They disappeared. Both shells use spaces and tabs only as separators between words: the xv6 shell skips any number of whitespace characters in gettoken() (the `whitespace` string on line 268 of sh.c), so the program received exactly three arguments, one, two and three, and none of them contains a space. bash does the same word splitting. To keep the spaces, the text must be quoted in bash ("one two three" gives one argument); in xv6 it cannot be done at all.

4 READ THE SHELL

4.1 The kinds of command

```
wc -l user/sh.c
grep -n "#define" user/sh.c
```

user/sh.c has **499 lines**. Lines 8–12 define the five kinds of command (Figure 4.1); MAXARGS 10 on line 14 is the maximum number of arguments of one command. The parser turns a command line into a tree of structures (execcmd, redircmd, pipecmd, listcmd, backcmd), and the type field of each node is one of these constants.

```
kynex@lenovo:~/xv6-riscv$
kynex@lenovo:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c
kynex@lenovo:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
kynex@lenovo:~/xv6-riscv$
```

Figure 4.1 – The command kinds defined in sh.c

4.2 Where each kind is run

```
grep -n "case " user/sh.c
```

The grep shows three groups of case labels (Figure 4.2):

- lines 75–125 – the switch in runcmd(), where each kind is executed: EXEC calls exec(), REDIR closes a file descriptor and opens the file in its place, LIST runs the left command in a child, waits, then runs the right one, PIPE creates a pipe and two children connected with dup(), BACK forks a child and does not wait for it;
- lines 284–297 – gettoken(), which recognizes the special characters | ( ) ; & < > and turns >> into the token '+';
- lines 391–397 – parseredirs(), which opens the file for <, > (with truncation) or >> (without truncation); and lines 469–493 – nulterminate(), which puts the 0 bytes at the ends of the words for every kind.

```
kynex@lenovo:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
kynex@lenovo:~/xv6-riscv$
```

Figure 4.2 – Where each kind is run

4.3 The code that handles cd

```
grep -n "cd" user/sh.c | head
```

The grep finds only line 170, the error message "cannot cd %s" (Figure 4.3). The test itself does not contain the text "cd", because it compares single characters. It is in main(), lines 166–171 of sh.c [2]:

```
if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
  // Chdir must be called by the parent, not the child.
  cmd[strlen(cmd) - 1] = 0; // chop \n
  if (chdir(cmd + 3) < 0)
    fprintf(2, "cannot cd %s\n", cmd + 3);
} else {
  if (fork1() == 0)
    runcmd(parsecmd(cmd));
  wait(0);
}
```

```
kynex@lenovo:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
kynex@lenovo:~/xv6-riscv$
```

Figure 4.3 – The code that handles cd

4.4 Answers to the Observe questions

1. Name the five kinds of command the xv6 shell knows and the character that produces each.

EXEC – a simple command, a program name with arguments (no special character); **REDIR** – < , > or >> ; **PIPE** – | ; **LIST** – ; ; **BACK** – & . Parentheses ( ) do not create a new kind: parseblock() only groups a sub-command, which is then used inside one of these five.

2. Where is cd handled, and why is it not run as a separate program?

cd is handled in main() of sh.c, lines 166–171, before the shell forks: if the line starts with "cd ", the shell calls chdir() itself. Every other command is run with fork1() + runcmd() in a child process. The current directory is a property of each process; if cd were a separate program (or were run in the child), chdir() would change only the directory of the child, which then exits, and the shell would stay where it was – this is what the comment "Chdir must be called by the parent, not the child" says. A consequence of this simple check is that cd only works when it is the first word of the line: in `mkdir d ; cd d` the cd is part of a LIST and is executed in a child, so it has no effect on the shell.

3. Search sh.c for any code that handles * or $. What does that explain about Part 2?

There is none. The only "$" in sh.c is the prompt `write(2, "$ ", 2)` on line 137, and "*" appears only in C pointer declarations. The only special characters are `symbols[] = "<|>&;()"` and the whitespace characters (lines 268–269); quotes are not handled either. This explains Part 2 and Part 3: `ls *` passed a literal * to ls ("cannot open *"), `echo $HOME` printed $HOME, and the quotes stayed in the arguments and did not join words. Globbing, variables and quote removal are all work that bash does before exec, and the xv6 shell simply does not do it.

CONCLUSIONS

In this laboratory work I compared bash with the xv6 shell, a complete shell of only 499 lines. First, I updated my sleep program from Lab 2 for the current xv6, where the system call is called pause(); the program builds, sleeps for the given number of ticks and prints its usage message when the argument is missing.

Running the same commands in the two shells showed what the minimum of a shell is: reading a line, splitting it into words, and then fork, exec, wait, plus redirection, pipes, lists, background commands and grouping. All of these worked the same in xv6 and in bash. Everything else – globbing, variables, quote removal, the PATH search, job control, history and completion – is added on top by bash. The error "exec ls failed" after cd d showed clearly that without PATH the program name is just a file path relative to the current directory.

The args program answered the main question of the lab: the program and the kernel only receive and pass on an array of strings, and it is the shell that decides what this array contains. With the same source code, bash gave the program args_linux.c and "x y", while xv6 gave it *.c, "x and y". Reading sh.c confirmed it: there is no code for * or $, only the symbols <|>&;() and whitespace. Finally, the code for cd showed why it must be a builtin: chdir() has to be called by the shell process itself, not by a child.

BIBLIOGRAPHY

1. Vasile Brînză. Lab work [online]. GitHub repository, 2026 [viewed 19 October 2026]. Available from: https://github.com/kynexi/os-2026-VasileBrinza

2. MIT PDOS. xv6-riscv [source code]. GitHub repository [viewed 19 October 2026]. Available from: https://github.com/mit-pdos/xv6-riscv